



Functional Programming

Looping Without Loops

Markos Vassilis, Mediterranean College, 2024 - 2025, Spring

Week 20

Loops XOR Functional == 1

Grouping



- Consider the following problem:

You are given a sequence of n numbers and you are asked to group them into groups (tuples) of consecutive numbers of size m (each). Assume that n is divided by m for your convenience.

- How would you address it?
- Assume that numbers are provided as a list (or a generator?) and that your function should return a list of tuples of size m .

Grouping: Attempt 1



```
def group_by(numbers, m):  
    """ Gets a list / generator of numbers and returns a list of tuples of groups of those  
    numbers of size m. """  
    n = len(numbers)  
    n_groups = n // m # Assuming n % m == 0  
    return [tuple(*[numbers[i * m:(i + 1) * m]]) for i in range(n_groups)]
```

- Any comments on that?
- What about resources?
 - Time?
 - Memory?

Grouping: Attempt 1



```
from psutil import Process # Maybe `pip install psutil` as this is not part of stdlib
import os # To access our own pid (Process ID)
import time # To time things
# group_by() defined here...
def test(fn, N = 10 ** 7, n = 2):
    NUMS = range(N) # Adjust this to run in a reasonable amount of time.
    start = time.process_time()
    groups = fn(NUMS, n)
    end = time.process_time()
    execution_time = end - start
    mem_consumption = Process(os.getpid()).memory_info().rss / 1024 ** 2 # Memory in MB
    print(f"Memory required: {mem_consumption} MB.")
    print(f"Execution time : {execution_time} seconds.")

if __name__ == "__main__":
    test(group_by)
```

Grouping: Attempt 1



- Typical output should look like this:

Memory required: 662.75390625 MB.

Execution time : 1.462404078 seconds.

- Aren't 600+ MBs a bit too much for such a simple task?
- How could we do better?
 - Maybe generators?
 - Maybe [itertools](#)?
- Experiment on your own for a bit!

Grouping: Attempt 2



```
def igroup_by(numbers, m):  
    inumbers = iter(numbers)  
    iters = [inumbers] * m  
    return zip(iters)
```

- What about (space / time) complexity?

Memory required: 11.515625 MB.

Execution time : 1.96599999999990796e-06 seconds.

On Windows time might be 0.0 (because they're really bad at reporting CPU clock values)

Grouping: Attempt 2



- How does this thing work?

```
def igroup_by(numbers, m):  
    inumbers = iter(numbers) # Create an iterator over those numbers  
    iters = [inumbers] * m # Create a list of `m` copies of the same iterator (?)  
    return zip(iters) # Functional sorcery takes place here..
```

- Okay, sooooo...?
- Take pen and paper and keep track of what happens in the background!

Iterators: Behind The Scenes



- So, what we do above is to create a list of m iterators. Assume for a moment that $m = 3$ and that we have 12 numbers:

```
numbers = [2, 1, 5, 8, 3, 0, 9, 11, 10, 4, 6, 7]
iters = [
    <iteratorObject (2, 1, 5, 8, 3, 0, 9, 11, 10, 4, 6, 7)>,
    <iteratorObject (2, 1, 5, 8, 3, 0, 9, 11, 10, 4, 6, 7)>,
    <iteratorObject (2, 1, 5, 8, 3, 0, 9, 11, 10, 4, 6, 7)>
]
```

- Actually, this is a bit wrong...
 - Can you see why?

Iterators: Behind The Scenes



- We actually create the iterator just once and then create a list of **references** to that one instance.
- So, more appropriately:

```
numbers = [2, 1, 5, 8, 3, 0, 9, 11, 10, 4, 6, 7]
iter = <iteratorObject (2, 1, 5, 8, 3, 0, 9, 11, 10, 4, 6, 7)>
iters = [
    <iteratorObject at 0x000456789123>,
    <iteratorObject at 0x000456789123>,
    <iteratorObject at 0x000456789123>
]
```

- Assuming, of course, that `iter` is kept at `0x000456789123`.

Iterators: Behind The Scenes



- So, what is the effect of zip in iters?
- zip starts by asking the first iterator to provide its next element (2), so:

```
iter = <iteratorObject (2, 1, 5, 8, 3, 0, 9, 11, 10, 4, 6, 7)>
```

```
iters = [  
    <iteratorObject at 0x000456789123>,  
    <iteratorObject at 0x000456789123>,  
    <iteratorObject at 0x000456789123>  
]
```

- So, the first zipped tuple is (so far): [(2),]

Iterators: Behind The Scenes



- Then, zip asks the second iterator to provide its next element (1), so:

```
iter = <iteratorObject (1, 5, 8, 3, 0, 9, 11, 10, 4, 6, 7)>
```

```
iters = [  
    <iteratorObject at 0x000456789123>,  
    <iteratorObject at 0x000456789123>,  
    <iteratorObject at 0x000456789123>  
]
```

- So, the first zipped tuple is (so far): [(2, 1),]

Iterators: Behind The Scenes



- Then, zip asks the third iterator to provide its next element (5), so:

```
iter = <iteratorObject (5, 8, 3, 0, 9, 11, 10, 4, 6, 7)>
iters = [
    <iteratorObject at 0x000456789123>,
    <iteratorObject at 0x000456789123>,
    <iteratorObject at 0x000456789123>
]
```

- So, the first zipped tuple is (so far): [(2, 1, 5),]

Iterators: Behind The Scenes



- Then, zip asks the first iterator to provide its next element (8), so:

```
iter = <iteratorObject (8, 3, 0, 9, 11, 10, 4, 6, 7)>
iters = [
    <iteratorObject at 0x000456789123>,
    <iteratorObject at 0x000456789123>,
    <iteratorObject at 0x000456789123>
]
```

- So, the second zipped tuple is (so far): [(2, 1, 5), (8,),]

Iterators: Behind The Scenes



- Then, zip asks the second iterator to provide its next element (3), so:

```
iter = <iteratorObject (3, 0, 9, 11, 10, 4, 6, 7)>
iters = [
    <iteratorObject at 0x000456789123>,
    <iteratorObject at 0x000456789123>,
    <iteratorObject at 0x000456789123>
]
```

- So, the second zipped tuple is (so far): [(2, 1, 5), (8, 3,),]

Iterators: Behind The Scenes



- Then, zip asks the second iterator to provide its next element (0), so:

```
iter = <iteratorObject (0, 9, 11, 10, 4, 6, 7)>
```

```
iters = [  
    <iteratorObject at 0x000456789123>,  
    <iteratorObject at 0x000456789123>,  
    <iteratorObject at 0x000456789123>  
]
```

- So, the second zipped tuple is (so far): [(2, 1, 5), (8, 3, 0),]
- And things proceed in a similar fashion...

Dealing Hands



- Imagine you are write a card-playing application, say for a Poker game.
- To that end, you will need to:
 - represent your deck, e.g., as a list or tuple or iterator;
 - shuffle your deck;
 - cut your deck at some arbitrary card;
 - deal your deck.
- Implement all of the above functionalities in a Python script.
 - Feel free to (and ashamed of) using loops, if needed.

Dealing Hands: The `itertools` Way



- To begin with, one can easily create a deck using the `it.product()` function, which computes the Cartesian product of all provided arguments:

```
import itertools as it
import random

ranks = ['A', 'K', 'Q', 'J', '10', '9', '8', '7', '6', '5', '4', '3', '2']
suits = ['H', 'D', 'C', 'S']

cards = it.product(ranks, suits)
```

Adapted from: <https://realpython.com/python-itertools/#dealing-a-deck-of-cards>

Dealing Hands: The `itertools` Way



- Regarding shuffling, one can just use `random.shuffle()`:
 - To that end, we have to turn our iterator into a list, since in order to shuffle uniformly randomly, we have to know the entire collection of the items to be shuffled.

```
def shuffle(deck):  
    deck = list(deck)  
    random.shuffle(deck) # Well, to shuffle randomly you need to know what you have to  
shuffle...  
    return iter(tuple(deck))
```

Dealing Hands: The `itertools` Way



- Then, what about cutting?
 - We essentially need a way to slice and chain iterators...
 - ...which `islice()` and `chain()` offer to us for free.

```
def cut(deck, n):  
    first, second = it.tee(deck, 2)  
    top = it.islice(first, n)  
    bottom = it.islice(second, n, None)  
    return it.chain(bottom, top)
```

Dealing Hands: The `itertools` Way



- Using some of the tools presented above, can you think about a purely iterator-based way to deal our shuffled and cut deck?

```
def deal(deck, num_hands=1, hand_size=5):  
    iters = [iter(deck)] * hand_size  
    return tuple(zip(*(tuple(it.islice(itr, num_hands)) for itr in iters)))  
  
hands = deal(cut(shuffle(cards), 30), num_hands=5)  
print("\n".join((str(h) for h in hands)))
```

Fun Time!

Do You Read Books?



In case you have not fully implemented last week's lab, you can find the instructions here:

`./labs/books_lab_01.pdf`

As this is relevant to your assignment, take care to work on all tasks, especially the ones related to the actual Goodreads dataset.

Any Questions



Don't forget to fill in the questionnaire shown right!



<https://forms.gle/YU1mjxjBjBf8hyvH7>